

Ada 2022 Attributes

A practical, developer-focused reference to the apostrophe-prefixed attributes used to query (and sometimes define) language semantics.

Updated: February 01, 2026

In Ada, an **attribute** is a characteristic of an entity that you query using an **attribute_reference** (for example, `T'First` or `A'Length(1)`). Some attributes also participate in representation and operational semantics (for example, `X'Address`, `T'Alignment`, or stream attributes like `T'Read`).

Syntax patterns

Basic: `Prefix'Attribute`

With argument(s): `Prefix'Attribute(Arg)` (e.g., `E'Pos(X)`, `A'Length(2)`)

Dimensioned array attributes: `A'First(D)`, `A'Last(D)`, `A'Range(D)`, `A'Length(D)`

Range attribute reference: `T'Range` or `A'Range(D)` (commonly used in loops and constraints)

Tip: Most attributes are *queries* (read-only). Some can be *specified* via an attribute definition clause (for example, specifying `Storage_Size` for an access type).

Ada 2022 highlights

'Image for any type: Ada 2022 generalizes 'Image (and Wide/Wide_Wide variants) beyond scalar types. You can also provide a user-defined image via 'Put_Image.

This is extremely handy for logging, debugging, and quick serialization-style display output without writing bespoke To_String functions for every type.

Example

```
type Vector is array (Positive range <>) of Integer;  
V : Vector := [1, 2, 3];  
Put_Line (V'Image);
```

Core attributes you will use constantly

Attribute	Applies to (prefix)	Returns	Notes / Example
'First	discrete types; arrays (dim)	index/first	For arrays use A'First(D). Common in loops.
'Last	discrete types; arrays (dim)	index/last	For arrays use A'Last(D).
'Range	discrete types; arrays (dim)	range	Often used in loops: for I in A'Range(1) loop.
'Length	arrays (dim)	Natural	Number of elements in a dimension: A'Length(1).
'Image	any type (Ada 2022)	String	Display-form string for a value. Wide variants return Wide_String / Wide_Wide_String.
'Value	scalar types	T	Parse a string into a value: T'Value("42").
'Size	types/objects	bits	Representation size in bits (context-dependent).
'Object_Size	types	bits	Size used for objects of the type (can differ from 'Size).
'Alignment	types/objects	bytes	Required alignment for objects of the type/object.
'Address	objects; subprograms	System.Address	Low-level address (representation/interop).
'Valid	objects	Boolean	Checks whether an object's representation is valid for its subtype (useful at boundaries).
'Constrained	subtypes/objects	Boolean	Whether a subtype or object is constrained (esp. arrays, discriminants).
'Class	tagged types	T'Class	Use for polymorphism: T'Class.
'Tag	tagged types/objects	tag	Runtime tag for dispatching / RTTI-like uses.

Discrete and enumeration attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'Pos	discrete types	universal_integer	Position number of a value (esp. enums): <code>Color'Pos(Red)</code> .
'Val	discrete types	T	Inverse of 'Pos: <code>Color'Val(0)</code> .
'Succ	discrete types	T	Successor: <code>Day'Succ(Mon)</code> .
'Pred	discrete types	T	Predecessor: <code>Day'Pred(Tue)</code> .
'Width	scalar types	universal_integer	Width needed for 'Image (mostly relevant for formatting).
'Enum_Rep	enum types	integer	Representation value for an enum literal (useful for interop).
'Enum_Val	enum types	T	Inverse of 'Enum_Rep.

Real and fixed-point attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'Digits	floating point types	universal_integer	Decimal digits of precision.
'Machine_Radix	floating point types	universal_integer	Radix used for machine representation.
'Machine_Exponent	floating point types	universal_integer	Minimum machine exponent.
'Machine_Maximum_Exponent	floating point types	universal_integer	Maximum machine exponent.
'Model_Small	floating/fixed types	universal_real	Smallest positive model number (model vs machine).
'Delta	fixed-point types	universal_real	Delta (step size) of a fixed-point type.
'Small	fixed-point types	universal_real	Smallest positive value (for some fixed types).
'Fore	fixed-point types	universal_integer	Default digits before decimal (Text_IO formatting).
'Aft	fixed-point types	universal_integer	Default digits after decimal (Text_IO formatting).
'Safe_First	real types	T	Safe first value representable (often for numeric safety margins).
'Safe_Last	real types	T	Safe last value representable.

Array and composite attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'First(D)	array objects/types	index	First index of dimension D.
'Last(D)	array objects/types	index	Last index of dimension D.
'Range(D)	array objects/types	range	Index range of dimension D.
'Length(D)	array objects/types	Natural	Number of components in dimension D.
'Component_Size	array types	bits	Size in bits for each component (representation control).

Access (pointer) and allocation-related attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'Access	objects/subprograms	access	Create an access value designating an aliased object or subprogram.
'Unchecked_Access	objects	access	Bypasses accessibility checks (unsafe; isolate usage).
'Storage_Size	access types; task types	universal_integer	Size of storage pool / task stack when specified; can be queried too.
'Storage_Pool	access types	pool	Refers to the storage pool used for allocations of an access type.
'Address	access objects too	System.Address	Low-level address; often paired with System and interfacing.

Reminder: in Ada, heap creation is called **allocation** (via `new T`), not instantiation. Allocation also performs initialization of the newly created object.

Stream-oriented attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'Read	types	procedure	Read a value of the type from a stream.
'Write	types	procedure	Write a value of the type to a stream.
'Input	types	function	Read and return a value (often includes bounds/discriminants).
'Output	types	procedure	Write a value (often includes bounds/discriminants).
'Stream_Size	types	universal_integer	Number of bits used by stream attributes for the type.

Tasking and entry attributes

Attribute	Applies to (prefix)	Returns	Notes / Example
'Callable	task objects	Boolean	True if a call to the task is allowed without raising <code>Tasking_Error</code> .
'Terminated	task objects	Boolean	True if the task has terminated.
'Count	entries	universal_integer	Number of callers currently queued on an entry.

Note: These are snapshots; in concurrent code, values can change immediately after you read them.

Implementation-defined attributes (GNAT and others)

In addition to language-defined attributes, compilers may provide extra attributes. GNAT documents these separately as implementation-defined attributes. Use them deliberately (and consider portability).

Examples you may encounter in GNAT

'Compiler_Version, 'Compilation_Time, 'From_Address, etc. (availability and semantics are compiler-specific).

References

Ada Reference Manual (2022): RM 4.1.4 (Attributes), RM 4.10 (Image Attributes), RM 9.9 (Task and Entry Attributes), RM 13.13.2 (Stream-Oriented Attributes).

AdaCore Learn: What's New in Ada 2022 - 'Image for any type, and related discussion of user-defined images.

GNAT Reference Manual: Implementation-defined attributes.